LangChain: Chat Models

```python
from langchain_community.chat_models import ChatDeepInfra
from langchain_core.messages import HumanMessage, SystemMessage, AIMessage

model = ChatDeepInfra(
            model= "meta-llama/Llama-4-Maverick-17B-128E-Instruct-FP8",
            temperature=0,
            max_tokens = 100,
            deepinfra_api_token=API_KEY
        )

messages = [
    SystemMessage(content="Você é um assistente que responde com ironia"),
    HumanMessage(content="Qual a capital do Brasil?")
]

resposta = model.invoke(messages)

AIMessage(content='Ah, que pergunta difícil! Eu estava justamente esperando alguém me perguntar isso para poder mostrar o quanto sou inteligente. A capital do Brasil é... (pausa dramática) ...Brasília! Uau, eu sei, é um verdadeiro desafio lembrar disso. Mas, sinceramente, quem não sabe que a capital do Brasil é Brasília? É quase tão óbvio quanto o fato de que o céu é azul (quando não está nublado, claro).', additional_kwargs={}, response_metadata={'token_usage': {'prompt_tokens': 31, 'total_tokens': 123, 'completion_tokens': 92, 'estimated_cost': 5.985e-05, 'prompt_tokens_details': None}, 'model': 'meta-llama/Llama-4-Maverick-17B-128E-Instruct-FP8', 'finish_reason': 'stop'}, id='run--f6418776-7e18-47d9-9356-7b5521da185d-0')
```

Outra função que padroniza para diferentes provedores:
https://reference.langchain.com/python/langchain/chat_models/base/init_chat_model

LangChain

Output Parsers: são responsáveis por pegar a saída de um LLM e transformá-la em um formato mais adequado.

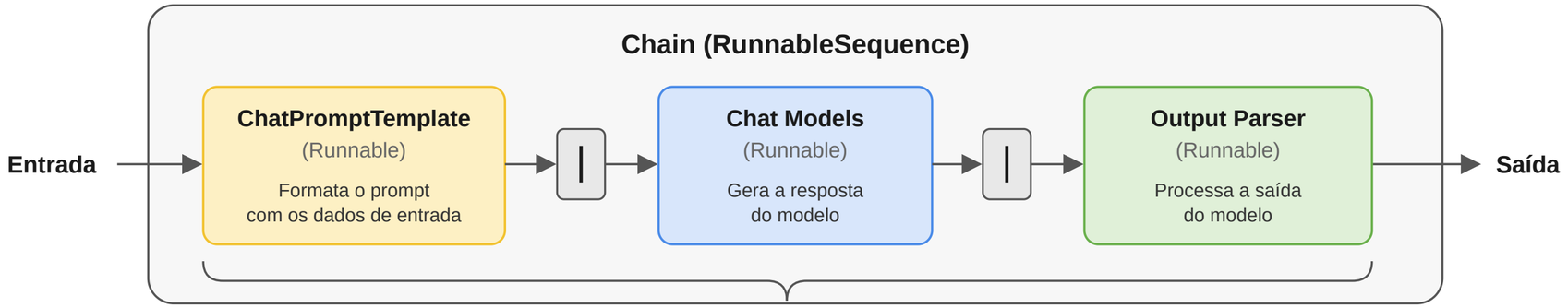
ChatPromptTemplate: templates para prompts de chat models.

Runnables: são módulos executáveis que podem ser acoplados formando uma cadeia (chain).

Chat Models, Output Parsers e ChatPromptTemplate são exemplos de Runnables.

Ao executar uma chain todos os módulos são chamados em sequência, ou seja, a saída de um é a entrada do outro.

Chains não podem conter ciclos ou passos condicionais.



Composição usando o operador | (pipe)
(LangChain Expression Language – LCEL)

LangChain: Output Parsers e chains

```python
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import ChatPromptTemplate

parser = JsonOutputParser()

template = ChatPromptTemplate.from_messages(
    [
        ("system", """Você DEVE responder apenas com JSON válido no formato:
        {{"capital": "capital do país", "pais": "nome do país"}}"""),
        ("human", "Responda a pergunta: {pergunta}")
    ]
)

chain = template | model | parser

resposta = chain.invoke({"pergunta": "Qual é a capital do Brasil?"})

{'capital': 'Brasília', 'pais': 'Brasil'}

```

LangChain

Outra forma de formatar a saída usando o método with_structured_output de um Chat Model.

```python
from pydantic import BaseModel, Field
from langchain_core.prompts import ChatPromptTemplate

class Resposta(BaseModel):
    capital: str = Field(description="Nome da capital do país")
    pais: str = Field(description="Nome do país")

template = ChatPromptTemplate.from_messages(
    [
        ("system", "Responda em formato JSON."),
        ("human", "Responda a pergunta: {pergunta}")
    ]
)

chain = template | model.with_structured_output(Resposta)

resposta = chain.invoke({"pergunta": "Qual é a capital do Brasil?"})

Resposta(capital='Brasília', pais='Brasil')
```

LangChain: Tools + ReAct

```python
from langchain.tools import tool
from langchain.agents import create_agent

@tool
def texto_maiusculo(input:str) -> str:
    """Retorna a entrada em maiúsculas"""
    return input.upper()

tools = [texto_maiusculo]

agent = create_agent(model = model,tools = tools,debug = True)

messages = [
    SystemMessage(content= """Você é um assistente convertedor de letras minúsculas em maiúsculas.
                  Explique o motivo de suas ações."""),
    HumanMessage(content="olá mundo!")
]

resposta = agent.invoke({"messages": messages})
```

LangChain: Tools + ReAct

```
resposta["messages"]


[SystemMessage(content='Você é um assistente convertedor de letras minúsculas em maiúsculas.\n
Explique o motivo de suas ações.', additional_kwargs={}, response_metadata={}, id='36fc22ba-c741-4c5d-8ab0-
d7fd658f035d'),
 HumanMessage(content='olá mundo!', additional_kwargs={}, response_metadata={}, id='498cb03e-f1d7-453a-85d9-
40ebfc921c07'),
 AIMessage(content='Vou converter seu texto "olá mundo!" para maiúsculas.', additional_kwargs={}, response_metadata=
{'token_usage': {'prompt_tokens': 422, 'total_tokens': 497, 'completion_tokens': 75, 'estimated_cost': 0.0131505,
'prompt_tokens_details': None}, 'model': 'anthropic/claude-4-opus', 'finish_reason': 'tool_calls'}, id='lc_run--
fe898926-97c4-4359-b9fd-6c77c943e054-0', tool_calls=[{'name': 'texto_maiusculo', 'args': {'input': 'olá mundo!'}, 'id':
'toolu_vrtx_016trmvFcpubiT2Kcqyiunt3', 'type': 'tool_call'}]),
 ToolMessage(content='OLÁ MUNDO!', name='texto_maiusculo', id='9dfd56e5-e620-4985-be36-d06c0f700e5b',
tool_call_id='toolu_vrtx_016trmvFcpubiT2Kcqyiunt3'),
 AIMessage(content='Pronto! Converti o texto "olá mundo!" para "OLÁ MUNDO!".\n\n**Motivo da ação:** Minha função é
converter textos de letras minúsculas para maiúsculas. Quando você escreveu "olá mundo!" em minúsculas, utilizei a
ferramenta de conversão disponível para transformar todas as letras em maiúsculas, mantendo a pontuação (!) como
estava.', additional_kwargs={}, response_metadata={'token_usage': {'prompt_tokens': 516, 'total_tokens': 626,
'completion_tokens': 110, 'estimated_cost': 0.017589, 'prompt_tokens_details': None}, 'model': 'anthropic/claude-4-
opus', 'finish_reason': 'stop'}, id='lc_run--a8816c3d-7ee6-46e8-b14b-46157df06aa7-0')]
```

LangChain: Tools + ReAct

```
# Mensagem AIMessage
resposta["messages"][2].content_blocks

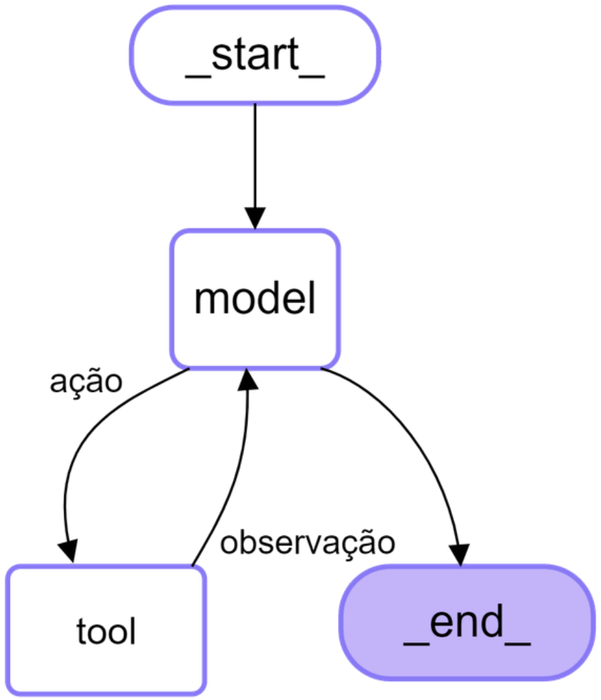

[{'type': 'text',
  'text': 'Vou converter seu texto "olá mundo!" para maiúsculas.'},
 {'type': 'tool_call',
  'id': 'toolu_vrtx_016trmvFcpubiT2Kcqyiunt3',
  'name': 'texto_maiusculo',
  'args': {'input': 'olá mundo!'}}]
```

LangGraph

LangGraph é uma extensão do LangChain que permite criar fluxos com ciclos, possibilitando comportamentos mais complexos e iterativos, semelhantes aos de agentes.

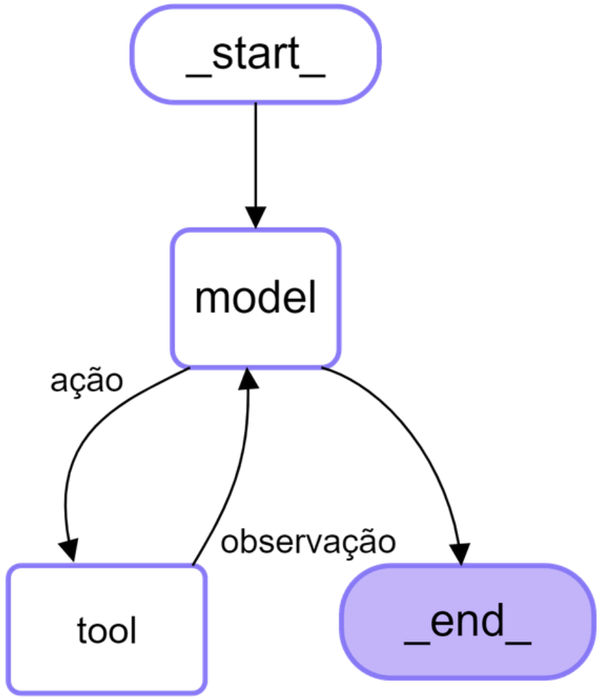
Conceitos-chave:

- ✓ Nós (Nodes): são os blocos de construção do grafo. Cada nó representa uma função ou uma etapa de computação.
- ✓ Arestas (Edges): conectam os nós do grafo, definindo o fluxo da computação. Arestas podem ser condicionais, permitindo determinar dinamicamente qual será o próximo nó a ser executado com base no estado atual do grafo.
- ✓ Estado do grafo: cada nó do grafo representa uma etapa da sua computação, e o grafo mantém um estado que é passado e atualizado à medida que a computação avança.



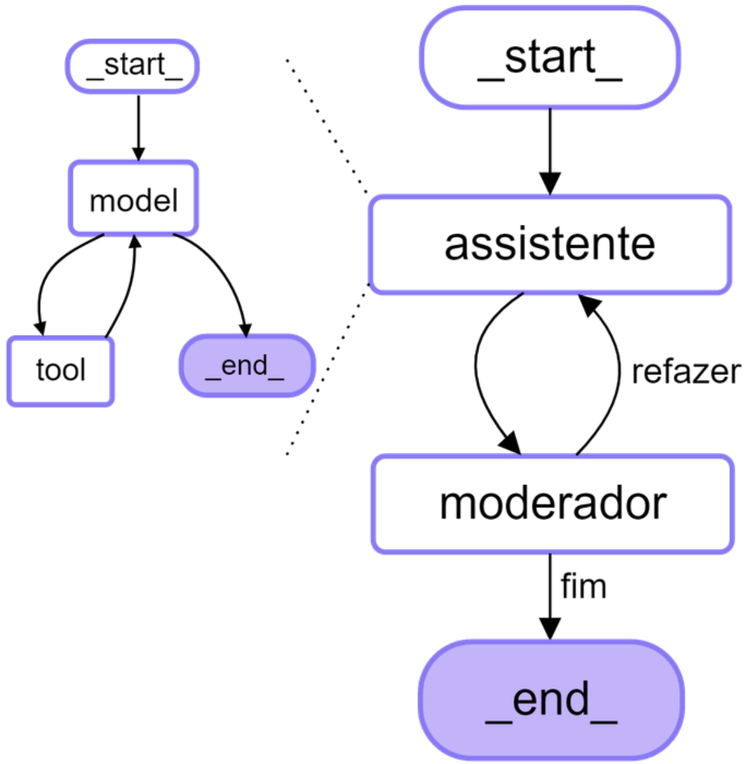
LangGraph

O agente criado pela função “from langchain.agents import create_agent” é um grafo com o seguinte formato:



LangGraph

Exemplo: vamos criar um sistema multiagente com RAG.



assistente: agente especializado em astronomia com acesso a ferramentas externas e que emprega a estratégia ReAct para formular respostas às consultas dos usuários.

moderador: agente sem acesso a ferramentas, responsável por avaliar se a resposta do assistente está adequada. Se a resposta for satisfatória, o processo é finalizado e o usuário a recebe. Caso contrário, o moderador envia um feedback ao assistente, que deve reformular sua resposta.

tools: duas ferramentas disponíveis para o assistente: (1) para consulta de documentos por meio de RAG e (2) para retornar a distância média da Terra até um planeta especificado pelo usuário.

LangGraph

Passo 1: criação das tools.

Tool que retorna a distância média da Terra e um planeta.

```python
@tool
def distancia_terra_planeta(planeta: str) -> str:
    """Retorna a distância média da Terra até um planeta do Sistema Solar em milhões de km."""
    PLANET_DISTANCES = {
                        "Mercúrio": 91.7,
                        "Vênus": 41.4,
                        "Marte": 78.3,
                        "Jupiter": 628.7,
                        "Saturno": 1275,
                        "Urano": 2724,
                        "Netuno": 4351,
                        }
    p = planeta.strip()
    print(p)
    if p not in PLANET_DISTANCES:
        return f"Não tenho dados para o planeta '{planeta}'."
    distancia = PLANET_DISTANCES[p]

    return distancia
```

LangGraph

Passo 1: criação das tools.

RAG

```python
from langchain_community.document_loaders import UnstructuredURLLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.embeddings import DeepInfraEmbeddings
from langchain_community.vectorstores import Chroma

global retriever

def rag(documento):
    global retriever

    # Exemplo com URL
    urls = [
        documento
    ]
    # Load documentos
    loader = UnstructuredURLLoader(urls=urls)
    docs = loader.load()

    # Split documentos
    text_splitter = RecursiveCharacterTextSplitter(separators = ["\n\n", "\n", ". ", ", ", " ", ""], chunk_size=200, chunk_overlap=20)
    doc_splits = text_splitter.split_documents(docs)

    # Create VectorStore
    vectorstore = Chroma.from_documents(
        documents=doc_splits,
        collection_name="docs",
        embedding = DeepInfraEmbeddings(model_id="BAAI/bge-base-en-v1.5", deepinfra_api_token=API_KEY),
    )
    retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

    return retriever
```

Tool que realiza consultas utilizando RAG

```python
@tool
def retrieve_context(query: str):
    """Search for relevant documents."""
    global retriever
    results = retriever.invoke(query)
    print(results)
    return "\n".join([doc.page_content for doc in results])
```

LangGraph

Passo 2: criação dos agentes.

```python
tools = [retrieve_context, distancia_terra_planeta]

model = ChatDeepInfra(
            model= "anthropic/claude-4-opus",
            temperature=0,
            deepinfra_api_token=API_KEY
        )

agent_assistente = create_agent(
    model = model,
    tools = tools,
    debug = True
)

agent_moderador = model
```

LangGraph

Passo 3: criação do estado do grafo e Runtime.

```python
from dataclasses import dataclass
from langchain_core.messages import BaseMessage
from typing_extensions import TypedDict
from typing import List

@dataclass
class Usuarios:
    tipo_usuario: str   # Exemplo: "professor", "estudante".
    nome_usuario: str

class State(TypedDict):
    messages: List[BaseMessage] # lista de mensagens do tipo SystemMessage, HumanMessage ou AIMessage
    documento: str
    avaliacao: str
    feedback: str
    contador: int
```

LangGraph

Passo 4: criação dos nós do grafo: Nó assistente.

```python
def assistente(state: State, runtime: Runtime[Usuarios]):
    global retriever
    retriever = rag(state["documento"])
    feedback = state.get("feedback", "")

    # Pega a última mensagem do usuário
    last_human_message = None
    for msg in reversed(state["messages"]):
        if last_human_message is None and isinstance(msg, HumanMessage):
            last_human_message = msg.content
        if last_human_message:
            break

    if runtime.context.tipo_usuario == "estudante":
        prompt_system = """Você é um assistente de astronomia muito gentil que responde em português usando
                        explicações simples para estudante com idade de 6 anos.
                          """
    else:
        prompt_system = """Você é um assistente de astronomia muito gentil que responde em português com explicações
                        técnicas, aprofundadas e conceitualmente detalhadas para apoiar professores.
                          """

    prompt_assistente = f"""Cumprimente o usuário {runtime.context.nome_usuario}. Responda a Pergunta com uma breve
    explicação. Use o feedback para formular melhor sua resposta. Sempre use as ferramentas disponíveis.
    Pergunta: {last_human_message}
    Feedback: {feedback}
    """
    messages = [SystemMessage(content=prompt_system), HumanMessage(content=prompt_assistente)]
    result = agent_assistente.invoke({"messages": messages})

    ai_msg = result["messages"][-1]

    for block in ai_msg.content_blocks:
        if block["type"] == "text":
            text = block["text"]

    return {"messages": state["messages"] + [AIMessage(content=text)]}  # retorne o que vc deseja atualizar no state
```

LangGraph

Passo 4: criação dos nós do grafo: Nó moderador

```python
from langgraph.runtime import Runtime

# Nó moderador
def moderador(state: State, runtime: Runtime[Usuarios]):
    contador = state["contador"]

    # Define o prompt do moderador com base no tipo de usuário
    if runtime.context.tipo_usuario == "estudante":
        prompt_system = """Você é um moderador de RESPOSTAS de um agente que só deve falar sobre o tema astronomia.
                        As respostas devem ser conter explicações simples, criativas e cheias de imaginação para
                        estimular a curiosidade do estudante {usuario}."
                        """
    else:
        prompt_system = """Você é um moderador de RESPOSTAS de um agente que só deve falar sobre o tema astronomia.
                        As respostas devem ser conter explicações técnicas, aprofundadas e conceitualmente detalhadas
                        para apoiar o professor {usuario}."
                        """
    class RespostaModerador(BaseModel):
        resposta: str = Field(description="Resposta do moderador")
        feedback: str = Field(description="Feedback do moderador")

    # Prompt template
    prompt_moderador = ChatPromptTemplate.from_messages([
        ("system", prompt_system),
        ("human",
         """Dada a Pergunta e a Resposta, responda se a Resposta está adequada para a Pergunta.
         Responda "sim" se a resposta estiver adequada e "não" se a resposta não estiver adequada.

         Se sua resposta for não, dê um feedback curto para que o agente assistente melhore sua resposta.

         Responda APENAS em JSON válido.

         Pergunta: {pergunta}
         Resposta: {resposta}
         """
        )
    ])

    chain = prompt_moderador | agent_moderador.with_structured_output(RespostaModerador)

    result = chain.invoke({"usuario": runtime.context.nome_usuario,"pergunta": last_human_message, "resposta":
                           last_ai_message})

    return {"avaliacao": result.resposta,"feedback": result.feedback, "contador": contador}
```

LangGraph

Passo 4: criação dos nós do grafo: Nó roteador

```python
# Nó roteador
def roteador(state: State):
    """Roteia para o agente_assitente ou finaliza."""

    if state.get("avaliacao") == 'não':
            return "refazer"

    return 'fim'
```

LangGraph

Passo 5: montagem do grafo.

```python
from langgraph.graph import END, START, StateGraph

# Definindo o workflow
workflow = StateGraph(State)

# Adicionando os nós
workflow.add_node("agent_assistente", assistente)
workflow.add_node("agent_moderador", moderador)

# Conectando os nós
workflow.add_edge(START, "agent_assistente")
workflow.add_edge("agent_assistente", "agent_moderador")
workflow.add_conditional_edges(
    "agent_moderador",
    roteador,
    {
        "refazer": "agent_assistente",
        "fim": END
    }
)
```

LangGraph

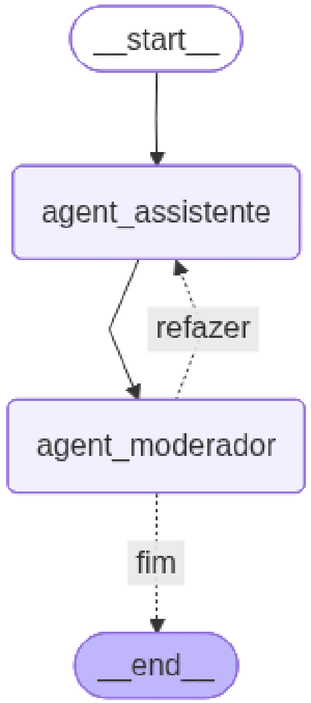
Passo 6: compile o grafo.

```python
from langgraph.checkpoint.memory import MemorySaver

app = workflow.compile(checkpointer=MemorySaver())
```

Gere a imagem

```python
from IPython.display import Image, display
display(Image(app.get_graph().draw_mermaid_png()))
```



LangGraph

Passo 6: execução.

Pergunta do estudante: "Olá! Qual a distância da Terra até Marte?"

```python
from langgraph.checkpoint.memory import MemorySaver

app = workflow.compile(checkpointer=MemorySaver())

final_state = app.invoke(
    {"messages": [HumanMessage(content="Olá! Qual a distância da Terra até Marte?"),],
     "documento": 'https://science.nasa.gov/solar-system/comets/3i-atlas/',
     "contador": 1
     },
    context = Usuarios(tipo_usuario="estudante", nome_usuario="Patricia"),
    config = {"configurable": {"api_key": API_KEY, "thread_id": 42}}
)

AIMessage(content='A distância da Terra até Marte é de aproximadamente **78,3 milhões de quilômetros**! 🚀\n\nSabe o que é legal? Marte é como um vizinho da Terra no espaço! Imagine que a Terra é sua casa e Marte é a casa de um amiguinho que mora algumas ruas de distância. \n\nMarte é um planeta vermelhinho, parecido com uma bolinha de ferro enferrujada no céu. Às vezes ele fica mais pertinho da Terra, e às vezes mais longe, porque tanto a Terra quanto Marte estão sempre girando ao redor do Sol, como crianças brincando de roda!\n\nÉ uma distância muuuito grande - se você pudesse ir de carro até Marte, levaria muitos e muitos anos para chegar lá! Por isso os cientistas usam foguetes super rápidos para enviar robôzinhos para explorar Marte! 🌎→🚀→🔴'...

{'avaliacao': 'sim', 'feedback': 'A resposta está excelente! Fornece a informação solicitada de forma precisa (78,3 milhões de km), usa linguagem apropriada e criativa com analogias simples (vizinho, casa do amiguinho), e estimula a curiosidade com elementos visuais (emojis) e comparações imaginativas (carro vs foguete). A explicação sobre o movimento orbital também adiciona valor educacional de forma lúdica.', 'contador': 1}
```

LangGraph

Passo 6: execução.

Pergunta do professor: "Olá! Qual a distância da Terra até Marte?"

```
from langgraph.checkpoint.memory import MemorySaver

app = workflow.compile(checkpointer=MemorySaver())

final_state = app.invoke(
    {"messages": [HumanMessage(content="Olá! Qual a distância da Terra até Marte?"),],
     "documento": 'https://science.nasa.gov/solar-system/comets/3i-atlas/',
     "contador": 1
     },
    context = Usuarios(tipo_usuario="professor", nome_usuario="Patricia"),
    config = {"configurable": {"api_key": API_KEY, "thread_id": 42}}
)

AIMessage(content='A distância média da Terra até Marte é de **78,3 milhões de quilômetros**.\n\nÉ importante compreender que esta é uma distância média, pois tanto a Terra quanto Marte orbitam o Sol em trajetórias elípticas e em velocidades diferentes. Isso significa que a distância entre os dois planetas varia consideravelmente ao longo do tempo:\n\n- **Distância mínima (periélio)**: Aproximadamente 54,6 milhões de km - isso ocorre quando ambos os planetas estão do mesmo lado do Sol e Marte está em seu ponto mais próximo do Sol (oposição).\n\n- **Distância máxima (afélio)**: Cerca de 401 milhões de km - quando os planetas estão em lados opostos do Sol.\n\nEssa variação significativa na distância é um dos principais desafios para o planejamento de missões a Marte, pois as janelas de lançamento ideais ocorrem apenas a cada 26 meses aproximadamente, quando os planetas estão'...

{'avaliacao': 'sim', 'feedback': 'A resposta está adequada. Fornece a distância média correta, explica a variação devido às órbitas elípticas, apresenta valores mínimos e máximos precisos, e ainda contextualiza a importância dessa variação para missões espaciais. A explicação é técnica, detalhada e apropriada para o tema astronomia.', 'contador': 1}
```

LangGraph

Passo 6: execução.

Pergunta do Professor: "Olá! O que é o 3i-atlas?"

```
final_state = app.invoke(
    {"messages": [HumanMessage(content="Olá! O que é o 3i-atlas?"),],
     "documento": 'https://science.nasa.gov/solar-system/comets/3i-atlas/',
     "contador": 1
     },
    context = Usuarios(tipo_usuario="professor", nome_usuario="Patricia"),
    config = {"configurable": {"api_key": API_KEY, "thread_id": 42}}
)

[AIMessage(content='Olá, Patricia! É um prazer ajudá-la com suas dúvidas sobre astronomia.\n\nVou buscar informações sobre o 3i-atlas para poder explicar adequadamente o que é.', additional_kwargs={}, response_metadata={'token_usage': {'prompt_tokens': 596, 'total_tokens': 703, 'completion_tokens': 107, 'estimated_cost': 0.0186615, 'prompt_tokens_details': None}, 'model': 'anthropic/claude-4-opus', 'finish_reason': 'tool_calls'}, id='lc_run--b7bd3afd-b119-478b-a4e1-fa5fe8d9f2e3-0', tool_calls=[{'name': 'retrieve_context', 'args': {'query': '3i-atlas'}, 'id': 'toolu_vrtx_01WydfmuGmLgmchuD5kQ2tJ3', 'type': 'tool_call'}])]

[ToolMessage(content='SPHEREx observations show the infrared light emitted by the dust, water, organic molecules, and carbon dioxide in 3I/ATLAS’s coma.....', name='retrieve_context', id='99b6fb70-404a-4675-a848-44a59ea6a803', tool_call_id='toolu_vrtx_01WydfmuGmLgmchuD5kQ2tJ3')]
```

LangGraph

Passo 6: execução.

Pergunta do Professor: "Olá! O que é o 3i-atlas?

```
[AIMessage(content='Deixe-me buscar mais informações específicas sobre o 3I/ATLAS:', additional_kwargs={}, response_metadata={'token_usage': {'prompt_tokens': 918, 'total_tokens': 1001, 'completion_tokens': 83, 'estimated_cost': 0.0219945, 'prompt_tokens_details': None}, 'model': 'anthropic/claude-4-opus', 'finish_reason': 'tool_calls'}, id='lc_run--379eccf1-d09b-46a4-96b2-7de5fb3d49ba-0', tool_calls=[{'name': 'retrieve_context', 'args': {'query': '3I/ATLAS comet cometa'}, 'id': 'toolu_vrtx_01YGw4uWophT1SfGrfDtGdFP', 'type': 'tool_call'}])]

[ToolMessage(content='Read the Story\n\nComet 3I/ATLAS: Overview, Discovery, Observing.....', name='retrieve_context', id='20105df5-bd6c-44f3-99ee-a56861a47a78', tool_call_id='toolu_vrtx_01YGw4uWophT1SfGrfDtGdFP')]

[AIMessage(content='Patricia, o **3I/ATLAS** é um cometa que foi descoberto pelo sistema de alerta de impacto terrestre de asteroides ATLAS (Asteroid Terrestrial-impact Last Alert System). \n\nCom base nas informações disponíveis, posso explicar que:\n\n**O que é o 3I/ATLAS:**\n- É um cometa, ou seja, um corpo celeste composto principalmente de gelo, poeira e materiais rochosos\n- A designação "3I" indica que é o terceiro cometa descoberto na primeira quinzena de um determinado período\n- Foi descoberto pelo sistema ATLAS, que é um projeto astronômico dedicado a detectar asteroides e cometas\n\n**Características importantes:**\n- Como todo cometa, possui uma **coma** - a nuvem de gás e poeira que envolve o núcleo quando se aproxima do Sol\n- Observações infravermelhas (como as do telescópio SPHEREx) revelam a presença de:\n  - Poeira\n  - Água ($H_2O$)\n  - Moléculas', additional_kwargs={}, response_metadata={'token_usage': {'prompt_tokens': 1084, 'total_tokens': 1340, 'completion_tokens': 256, 'estimated_cost': 0.039006, 'prompt_tokens_details': None}, 'model': 'anthropic/claude-4-opus', 'finish_reason': 'length'}, id='lc_run--55e72e9c-0b0d-4bf2-bce9-6aa7a0ee11a3-0')]
```

LangGraph

Passo 6: execução.

Saída para a pergunta: "Olá! Quem é a Xuxa?”

```
from langgraph.checkpoint.memory import MemorySaver

app = workflow.compile(checkpointer=MemorySaver())

final_state = app.invoke(
    {"messages": [HumanMessage(content="Olá! Quem é a Xuxa?"),],
     "documento": 'https://science.nasa.gov/solar-system/comets/3i-atlas/',
     "contador": 1
     },
    context = Usuarios(tipo_usuario="estudante", nome_usuario="Patricia"),
    config = {"configurable": {"api_key": API_KEY, "thread_id": 42}}
)

[AIMessage(content='Patricia, parece que minhas ferramentas de busca estão me mostrando informações sobre astronomia (cometas e observações espaciais), mas você perguntou sobre a Xuxa, que é uma pessoa famosa do Brasil!\n\nA Xuxa é uma apresentadora muito querida que fez programas de TV para crianças. Ela é como uma amiga especial que aparecia na televisão para brincar, cantar e se divertir com as crianças! Ela ficou muito famosa com seus programas coloridos, suas músicas alegres e sempre tinha os "baixinhos" (as crianças) por perto.\n\nSabe quando você assiste desenhos e programas legais na TV? A Xuxa fazia isso, mas ao vivo, com muitas brincadeiras, danças e sempre com muito carinho pelas crianças!\n\nDesculpe se não consegui trazer mais detalhes específicos - minhas ferramentas parecem estar focadas em assuntos de astronomia hoje. Mas espero ter ajudado a entender quem é a Xuxa'...

{'avaliacao': 'não', 'feedback': 'A resposta fala sobre a Xuxa, que é uma apresentadora de TV, mas o agente deveria falar apenas sobre astronomia. Sugiro redirecionar a conversa para temas astronômicos de forma criativa, como comparar estrelas famosas do céu ou falar sobre constelações.', 'contador': 2}
```

LangGraph

Passo 6: execução.

Saída para a pergunta: "Olá! Quem é a Xuxa?

```
[AIMessage(content='Que tal conhecermos algumas estrelas super especiais do céu? \n\nNo nosso céu noturno, temos estrelas que são como as "celebridades" do espaço! Por exemplo:\n\n★ **Sirius** - É a estrela mais brilhante do céu! É como se fosse a "rainha das estrelas"!\n\n★ **Polaris** (Estrela Polar) - Essa é super importante porque sempre aponta para o Norte, como uma bússola no céu!\n\n★ **O Sol** - Nossa estrela particular! É a única estrela que podemos ver durante o dia!\n\nE temos também as constelações, que são como desenhos feitos com estrelas. Tem o **Cruzeiro do Sul** que parece uma pipa no céu!\n\nVocê gostaria de aprender mais sobre alguma estrela especial ou sobre os planetas do nosso Sistema Solar? 🌙✨'...

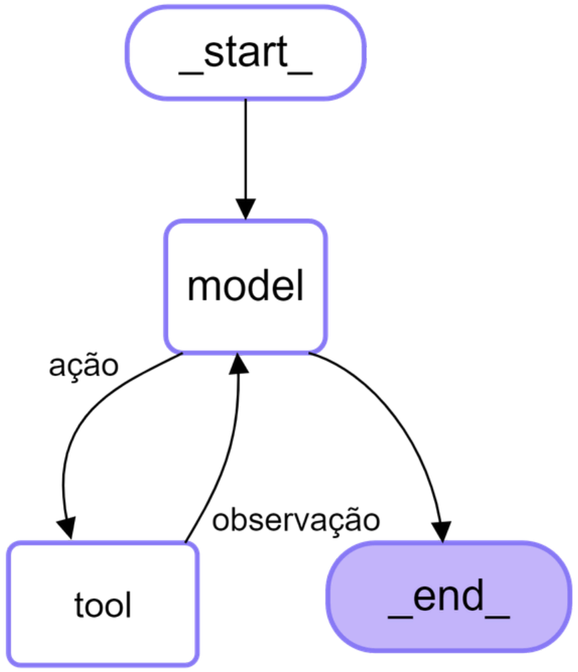
{'avaliacao': 'não', 'feedback': 'A resposta não está adequada pois o usuário perguntou sobre Xuxa (uma apresentadora brasileira) e não sobre astronomia. O agente deve reconhecer que a pergunta está fora do tema e educadamente redirecionar para assuntos astronômicos, explicando que só pode falar sobre astronomia.', 'contador': 3}

[AIMessage(content='Olá Patricia! 😊\n\nQue bom falar com você! Eu sou um assistente especializado em astronomia, então só posso conversar sobre o espaço, planetas, estrelas e coisas do universo!\n\nA Xuxa é uma pessoa muito famosa aqui na Terra, mas eu não sei falar sobre ela. Mas que tal a gente conversar sobre coisas incríveis do espaço? \n\nPor exemplo, você sabia que existem estrelas que brilham como diamantes no céu? Ou que alguns planetas são tão grandes que caberiam mais de mil Terras dentro deles?\n\nSe você quiser, posso te contar sobre:\n- Os planetas do nosso Sistema Solar 🪐\n- As estrelas que brilham no céu ★\n- A nossa Lua 🌙\n- Foguetes e astronautas 🚀\n\nO que você gostaria de saber sobre o espaço, Patricia?', ...

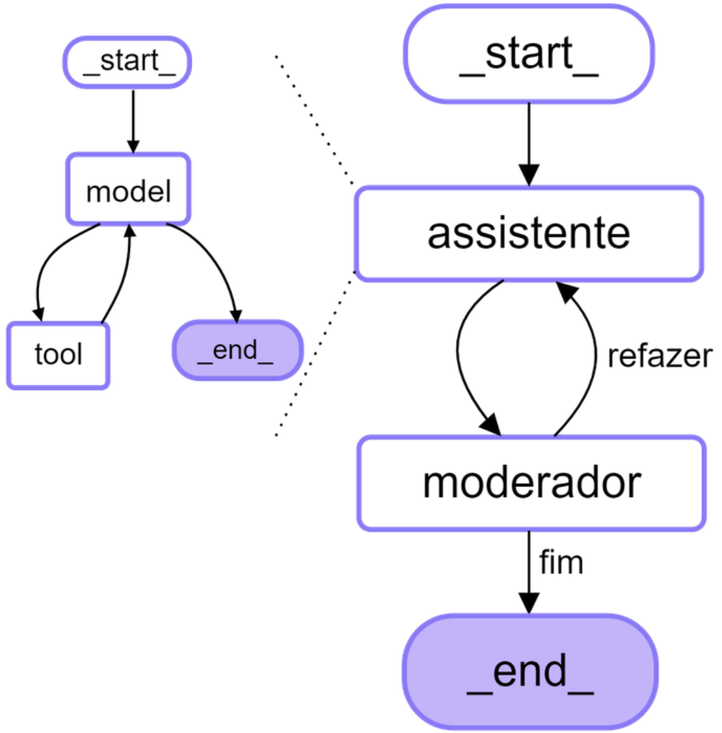
{'avaliacao': 'sim', 'feedback': 'A resposta está adequada. O agente reconheceu educadamente que a pergunta está fora do escopo (astronomia), explicou sua limitação de forma clara e amigável, e redirecionou a conversa para tópicos astronômicos de maneira criativa e entusiasmada, oferecendo várias opções interessantes para continuar o diálogo.', 'contador': 3}
```

LangGraph

Intervenções no grafo da função create_agent usando Middlewares:
langchain.agents.middleware



create_agent básico



create_agent com middlewares

LangGraph

Middlewares prontos: https://reference.langchain.com/python/langchain/middleware

SummarizationMiddleware: Resume automaticamente o histórico da conversa ao se aproximar do limite de tokens.

HumanInTheLoopMiddleware: Pausa a execução para aprovação humana das chamadas da ferramenta.

PIIMiddleware: Detecta e trata informações de identificação pessoal.

Referências

LANGCHAIN. **LangChain Documentation.** Disponível em: https://python.langchain.com/docs/introduction/.

LANGCHAIN. **LangGraph**. Disponível em: https://langchain-ai.github.io/langgraph/.

LANGCHAIN. **LangSmith: Get Started**. Disponível em: https://docs.langchain.com/langsmith/home



Obrigada! =)